

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить решение систем линейных уравнений методом Гаусса, реализовать программу с выбором ведущего элемента, проверить корректность её работы на базовых, специальных и больших тестах, а также научиться определять наличие решения системы и выявлять вырожденность матрицы в процессе вычислений.

Теоретический минимум.

Метод Гаусса применяется для решения системы линейных уравнений $A \cdot x = b$ и основан на последовательном исключении неизвестных с помощью элементарных преобразований строк расширенной матрицы $[A|b]$. На прямом ходе матрица приводится к ступенчатому виду: в каждом столбце выбирается ведущий элемент, после чего элементы ниже него зануляются. Затем на обратном ходе найденная ступенчатая система решается снизу вверх обратной подстановкой.

В практической реализации важно использовать выбор ведущего элемента, обычно частичный выбор по столбцу: на каждом шаге среди элементов текущего столбца выбирают максимальный по модулю и при необходимости меняют строки местами. Это нужно для того, чтобы избежать деления на ноль или на слишком маленькие числа и тем самым повысить численную устойчивость вычислений.

Метод Гаусса позволяет не только найти решение, но и определить, существует ли оно вообще. После прямого хода анализируются ранги матрицы коэффициентов A и расширенной матрицы $[A|b]$. Если $\text{rank}(A) = \text{rank}([A|b]) = n$, то система имеет единственное решение и матрица невырождена. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, то система совместна, но имеет бесконечно много решений, а матрица вырождена. Если $\text{rank}(A) < \text{rank}([A|b])$, то система несовместна и решений не имеет. Таким образом, вырожденность матрицы определяется по условию $\text{rank}(A) < n$.

Особое значение имеет контроль вырожденности в процессе работы алгоритма. Если на очередном шаге в рассматриваемом столбце не удаётся найти ненулевой ведущий элемент, это означает, что новый независимый *pivot* получить нельзя, следовательно, ранг матрицы меньше полного и матрица является вырожденной. Поэтому метод Гаусса одновременно решает систему и позволяет установить её структуру: единственное решение, бесконечно много решений или отсутствие решений.

С вычислительной точки зрения классический метод Гаусса для плотной матрицы имеет сложность порядка $O(n^3)$ по времени и $O(n^2)$ по памяти, поэтому для очень больших размеров он становится тяжёлым. Для проверки программы часто используют обычные системы с известным ответом, вырожденные и несовместные случаи, а также специальные плохо обусловленные матрицы, например матрицы Гильберта, которые хорошо показывают влияние ошибок округления и важность выбора ведущего элемента.

Выполнение работы.

В ходе выполнения работы была реализована программа на языке C++, предназначенная для решения системы линейных уравнений методом Гаусса с выбором ведущего элемента. В качестве способа выбора ведущего элемента был использован частичный выбор по столбцу: на каждом шаге прямого хода в текущем столбце среди всех доступных строк ищется элемент с максимальным модулем, после чего соответствующая строка при необходимости меняется местами с текущей. Такой подход позволяет избежать деления на ноль и уменьшает влияние вычислительных погрешностей.

На первом этапе программы выполняется проверка корректности входных данных. Контролируется, что матрица коэффициентов является квадратной, а размер вектора правых частей совпадает с числом строк матрицы. После этого по исходной системе формируется расширенная матрица вида $[A|b]$, с которой далее и работает алгоритм.

Основная часть решения реализована через прямой ход метода Гаусса. Для каждого столбца последовательно выбирается ведущий элемент. Если в рассматриваемом столбце среди оставшихся строк не удаётся найти ненулевой элемент, то этот столбец не может дать новый ведущий элемент, что уже в процессе вычислений указывает на уменьшение ранга матрицы и возможную вырожденность системы. Если ведущий элемент найден, строка с ним переносится на текущую позицию, после чего из всех строк ниже вычитается текущая строка, умноженная на соответствующий коэффициент. В результате элементы под ведущим элементом зануляются, а матрица постепенно приводится к ступенчатому виду.

После завершения прямого хода программа анализирует полученную ступенчатую форму и определяет ранги матрицы коэффициентов A и расширенной матрицы $[A|b]$. По этим значениям автоматически определяется тип системы. Если $\text{rank}(A) = \text{rank}([A|b]) = n$, то система имеет единственное решение, а матрица коэффициентов невырождена. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, то система имеет бесконечно много решений, а матрица является вырожденной.

Если $\text{rank}(A) < \text{rank}([A|b])$, то система несовместна и решений не имеет. Таким образом, программа не только находит решение, но и проверяет его существование, а также контролирует вырожденность матрицы непосредственно в ходе работы алгоритма.

Если после анализа рангов выясняется, что решение существует, программа выполняет обратный ход метода Гаусса. На этом этапе неизвестные вычисляются снизу вверх с помощью обратной подстановки. Для случая единственного решения находятся все значения переменных. Для случая бесконечного числа решений программа строит одно частное решение, полагая свободные переменные равными нулю.

Для удобства использования решение было разделено на несколько частей. В основном модуле реализован сам алгоритм метода Гаусса, включая проверку входных данных, выбор ведущего элемента, прямой ход, вычисление рангов и обратную подстановку. В отдельном файле `main.cpp` организован ввод исходной системы, вызов решателя и вывод полученного результата. Также был создан файл `test.cpp`, содержащий набор тестов для проверки корректности работы программы.

В процессе тестирования были рассмотрены несколько типов входных данных. Сначала были проверены базовые случаи с небольшими матрицами, для которых заранее известно правильное решение. Это позволило убедиться в правильности прямого и обратного хода, а также в корректности выбора ведущего элемента. Далее были протестированы вырожденные и несовместные системы. В этих примерах программа должна была не просто завершиться, а правильно определить, имеет ли система бесконечно много решений или не имеет решений вовсе. Такие тесты подтвердили корректность проверки рангов и реализации условия совместности системы.

Отдельно были проведены тесты на специальных матрицах Гильберта. Эти матрицы являются плохо обусловленными и используются для проверки устойчивости численных методов. Для тестирования задавалось известное решение, по нему вычислялся вектор правых частей, после чего система

решалась программой. Корректность проверялась по величине невязки, то есть по тому, насколько близко найденное решение удовлетворяет исходной системе. Эти тесты показали, что использование выбора ведущего элемента действительно повышает надёжность вычислений.

Кроме того, были предусмотрены проверки на матрицах больших размеров. Для размера 1000 x 1000 использовались простые невырожденные матрицы, например диагональные, для которых можно легко проверить правильность результата. Это позволило оценить работоспособность программы на больших объёмах данных. Также был предусмотрен тест размера 10000 x 10000, однако для плотной матрицы такой случай является крайне тяжёлым по памяти и времени, поскольку классический метод Гаусса имеет сложность $O(n^3)$. Поэтому в тестах такой случай разумно запускать только для специальных простых матриц или оставлять как отдельную проверку при необходимости.

В результате выполнения работы была создана программа, которая реализует решение СЛУ методом Гаусса с частичным выбором ведущего элемента, определяет наличие решения, проверяет невырожденность матрицы в процессе вычислений и проходит тестирование на базовых, специальных и больших примерах. Полученная реализация соответствует поставленным требованиям и может использоваться как для учебных, так и для практических задач небольшого и среднего размера.

Тестирование.

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод.

В ходе выполнения работы был изучен метод Гаусса для решения систем линейных уравнений и реализована программа на языке C++ с частичным выбором ведущего элемента. Разработанная программа позволяет не только находить решение системы, но и определять, существует ли оно, является ли единственным, а также проверять вырожденность матрицы коэффициентов в процессе вычислений. В результате тестирования на базовых примерах, вырожденных и несовместных системах, а также на специальных матрицах Гильберта и матрицах больших размеров было подтверждено, что программа корректно реализует алгоритм, правильно определяет тип системы и обеспечивает устойчивую работу за счёт выбора ведущего элемента. Таким образом, цель работы достигнута, а реализованный метод показал свою эффективность для решения СЛУ и анализа свойств матрицы системы.

ПРИЛОЖЕНИЕ А

Название файла: gauss.hpp

```
#pragma once

#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <stdexcept>
#include <string>
#include <vector>

// Перечисление для типа решения системы.
enum class SolutionType {
    Unique,    // Единственное решение
    Infinite,  // Бесконечно много решений
    None       // Решений нет
};

// Структура с результатом работы метода Гаусса.
struct GaussResult {
    SolutionType type = SolutionType::None;

    // Вектор решения.
    // Для Unique здесь будет единственное решение.
    // Для Infinite здесь можно хранить одно частное решение
    // (свободные переменные положены равными нулю).
    // Для None вектор будет пустым.
    std::vector<double> x;

    // Ранг матрицы коэффициентов A.
    int rankA = 0;

    // Ранг расширенной матрицы [A | b].
    int rankAugmented = 0;

    // Признак вырожденности матрицы A.
    // Если rankA < n, то матрица вырождена.
    bool isDegenerate = false;
};

// Класс, реализующий решение СЛУ методом Гаусса.
class GaussianSolver {
public:
    // Конструктор.
    // eps - допустимая погрешность для сравнения с нулём.
    explicit GaussianSolver(double eps = 1e-12) : eps_(eps) {}
```

```

// Основной метод решения системы  $A * x = b$ .
//
// A - квадратная матрица размера n x n
// b - вектор правых частей размера n
//
// Метод:
// 1) Проверяет корректность входных данных
// 2) Строит расширенную матрицу  $[A | b]$ 
// 3) Выполняет прямой ход Гаусса
// 4) Во время прямого хода фактически проверяет
//     невырожденность матрицы:
//     если в очередном столбце не найден ведущий элемент,
//     значит ранг матрицы меньше n
// 5) После прямого хода определяет:
//     - есть ли решение,
//     - единственно ли оно,
//     - или решений бесконечно много
// 6) При необходимости делает обратный ход

GaussResult solve(const std::vector<std::vector<double>>& A,
                  const std::vector<double>& b) const
{
    validateInput(A, b);

    const int n = static_cast<int>(A.size());

    // Строим расширенную матрицу  $[A | b]$ .
    std::vector<std::vector<double>> aug(n, std::vector<double>(n +
1, 0.0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            aug[i][j] = A[i][j];
        }
        aug[i][n] = b[i];
    }

    // row - номер текущей строки, в которую будем ставить
    // очередной ведущий элемент.
    int row = 0;

    // Прямой ход метода Гаусса.
    // Перебираем столбцы слева направо.
    for (int col = 0; col < n && row < n; ++col) {
        // Ищем строку с максимальным по модулю элементом
        // в текущем столбце среди строк row..n-1.
        int pivotRow = selectPivotRow(aug, row, col);

        // Если в этом столбце нет ненулевого элемента,
        // то ведущий элемент поставить нельзя.
        // Это означает, что столбец не даёт новый pivot,
        // а матрица потенциально вырождена.
        if (pivotRow == -1) {
            continue;
        }

        // Если нужно, меняем строки местами,
        // чтобы pivot оказался в строке row.
        if (pivotRow != row) {

```

```

        std::swap(aug[pivotRow], aug[row]);
    }

    // Обнуляем элементы ниже ведущего.
    eliminateBelow(aug, row, col);

    // Переходим к следующей строке.
    ++row;
}

// После прямого хода вычисляем ранги матрицы A
// и расширенной матрицы [A | b].
const auto [rankA, rankAugmented] = computeRanks(aug);

GaussResult result;
result.rankA = rankA;
result.rankAugmented = rankAugmented;
result.isDegenerate = (rankA < n);

// Если rank(A) < rank([A|b]), то система несовместна.
if (rankA < rankAugmented) {
    result.type = SolutionType::None;
    return result;
}

// Если rank(A) = rank([A|b]) < n, то решений бесконечно много.
// В этом случае построим одно частное решение:
// свободные переменные положим равными нулю.
if (rankA < n) {
    result.type = SolutionType::Infinite;
    result.x = buildOneSolutionFromEchelon(aug);
    return result;
}

// Если rank(A) = rank([A|b]) = n, то решение единственно.
result.type = SolutionType::Unique;
result.x = buildOneSolutionFromEchelon(aug);
return result;
}

private:
    double eps_;

    // Проверка "почти ноль".

    bool isZero(double value) const {
        return std::fabs(value) < eps_;
    }

    // Проверка корректности входных данных.
    //
    // Требования:
    // - A должна быть квадратной
    // - размер b должен совпадать с числом строк A

    void validateInput(const std::vector<std::vector<double>>& A,

```

```

        const std::vector<double>& b) const
    {
        if (A.empty()) {
            throw std::invalid_argument("Матрица A не должна быть
пустой.");
        }

        const std::size_t n = A.size();

        if (b.size() != n) {
            throw std::invalid_argument("Размер вектора b не совпадает с
размером матрицы A.");
        }

        for (std::size_t i = 0; i < n; ++i) {
            if (A[i].size() != n) {
                throw std::invalid_argument("Матрица A должна быть
квадратной.");
            }
        }
    }

    // Выбор ведущего элемента.
    //
    // Метод выбора:
    // частичный выбор по столбцу.
    //
    // Среди строк startRow..n-1 ищем строку с максимальным
    // по модулю элементом в столбце col.
    //
    // Если все элементы в этом столбце близки к нулю,
    // возвращаем -1.

    int selectPivotRow(const std::vector<std::vector<double>>& aug,
                      int startRow,
                      int col) const
    {
        const int n = static_cast<int>(aug.size());

        int bestRow = -1;
        double bestAbsValue = 0.0;

        for (int i = startRow; i < n; ++i) {
            const double currentAbsValue = std::fabs(aug[i][col]);
            if (currentAbsValue > bestAbsValue) {
                bestAbsValue = currentAbsValue;
                bestRow = i;
            }
        }

        if (bestRow == -1 || isZero(bestAbsValue)) {
            return -1;
        }

        return bestRow;
    }
}

```

```

// Прямой ход: обнуление элементов ниже ведущего.
//
// Пусть pivot находится в позиции (pivotRow, pivotCol).
// Тогда для каждой строки i > pivotRow:
//
// row_i = row_i - factor * row_pivot,
// где factor = aug[i][pivotCol] / aug[pivotRow][pivotCol]
//
// Мы специально пропускаем строки, где элемент уже близок к нулю,
// чтобы не делать лишнюю работу.

void eliminateBelow(std::vector<std::vector<double>>& aug,
                    int pivotRow,
                    int pivotCol) const
{
    const int n = static_cast<int>(aug.size());
    const double pivot = aug[pivotRow][pivotCol];

    for (int i = pivotRow + 1; i < n; ++i) {
        if (isZero(aug[i][pivotCol])) {
            // Если под pivot уже стоит 0,
            // то ничего делать не нужно.
            continue;
        }

        const double factor = aug[i][pivotCol] / pivot;

        // После вычитания этот элемент должен стать нулём.
        aug[i][pivotCol] = 0.0;

        // Изменяем элементы справа от pivot.
        for (int j = pivotCol + 1; j <= n; ++j) {
            aug[i][j] -= factor * aug[pivotRow][j];

            // Малые вычислительные шумы прижимаем к нулю,
            // чтобы матрица оставалась "чище".
            if (isZero(aug[i][j])) {
                aug[i][j] = 0.0;
            }
        }
    }
}

// Вычисление рангов матрицы A и расширенной матрицы [A | b]
// по уже приведённой ступенчатой форме.
//
// Для каждой строки:
// - если есть ненулевой коэффициент среди столбцов 0..n-1,
//   то эта строка увеличивает rank(A)
// - если в строке есть ненулевой коэффициент среди 0..n,
//   то она увеличивает rank([A|b])
//
// Особенно важен случай:
// [0 0 0 ... 0 | c], где c != 0
// Тогда rank([A|b]) > rank(A), и система несовместна.

```

```

std::pair<int, int> computeRanks(const
std::vector<std::vector<double>>& aug) const
{
    const int n = static_cast<int>(aug.size());

    int rankA = 0;
    int rankAugmented = 0;

    for (int i = 0; i < n; ++i) {
        bool hasNonZeroInA = false;
        bool hasNonZeroInAugmented = false;

        for (int j = 0; j < n; ++j) {
            if (!isZero(aug[i][j])) {
                hasNonZeroInA = true;
                hasNonZeroInAugmented = true;
                break;
            }
        }

        // Если в коэффициентах всё нули, проверяем свободный член.
        if (!hasNonZeroInA && !isZero(aug[i][n])) {
            hasNonZeroInAugmented = true;
        }

        if (hasNonZeroInA) {
            ++rankA;
        }
        if (hasNonZeroInAugmented) {
            ++rankAugmented;
        }
    }

    return {rankA, rankAugmented};
}

// Находит индекс первого ненулевого коэффициента в строке
// среди столбцов 0..n-1.
//
// Если таких нет, возвращает -1.

int firstNonZeroColumn(const std::vector<double>& row) const
{
    const int n = static_cast<int>(row.size()) - 1; // последний
столбец - это b
    for (int j = 0; j < n; ++j) {
        if (!isZero(row[j])) {
            return j;
        }
    }
    return -1;
}

// Построение одного решения из ступенчатой формы.
//
// Этот метод работает и для случая единственного решения,

```

```

// и для случая бесконечного числа решений.
//
// Идея:
// - свободные переменные оставляем равными нулю
// - для ведущих переменных идём снизу вверх
// и выполняем обратную подстановку

std::vector<double> buildOneSolutionFromEchelon(
    const std::vector<std::vector<double>>& aug) const
{
    const int n = static_cast<int>(aug.size());
    std::vector<double> x(n, 0.0);

    // Идём снизу вверх по строкам.
    for (int i = n - 1; i >= 0; --i) {
        const int leadCol = firstNonZeroColumn(aug[i]);

        // Полностью нулевая строка пропускается.
        if (leadCol == -1) {
            continue;
        }

        // Считаем сумму уже найденных членов справа.
        double sum = aug[i][n];
        for (int j = leadCol + 1; j < n; ++j) {
            sum -= aug[i][j] * x[j];
        }

        x[leadCol] = sum / aug[i][leadCol];
    }

    return x;
}

};

// Вспомогательная функция печати решения.
// Не обязательна для алгоритма, но удобна в main.cpp.

inline void printGaussResult(const GaussResult& result)
{
    std::cout << std::setprecision(15);

    std::cout << "rank(A) = " << result.rankA << "\n";
    std::cout << "rank([A|b]) = " << result.rankAugmented << "\n";
    std::cout << "Матрица A "
        << (result.isDegenerate ? "вырождена" : "невырождена")
        << "\n";

    if (result.type == SolutionType::None) {
        std::cout << "Система несовместна: решений нет.\n";
        return;
    }

    if (result.type == SolutionType::Infinite) {
        std::cout << "Система имеет бесконечно много решений.\n";
        std::cout << "Одно частное решение (свободные переменные =
0):\n";

```



```

    } else {
        std::cout << "Система имеет единственное решение:\n";
    }

    for (std::size_t i = 0; i < result.x.size(); ++i) {
        std::cout << "x[" << i << "] = " << result.x[i] << "\n";
    }
}

```

Название файла: main.cpp

```

#include "gauss.hpp"

#include <iostream>
#include <vector>

// Формат ввода:
//
// n
// a11 a12 ... a1n
// a21 a22 ... a2n
// ...
// an1 an2 ... ann
// b1 b2 ... bn

int main()
{
    try {
        int n;
        std::cin >> n;

        if (!std::cin || n <= 0) {
            std::cerr << "Ошибка: некорректный размер матрицы.\n";
            return 1;
        }

        std::vector<std::vector<double>> A(n, std::vector<double>(n));
        std::vector<double> b(n);

        // Считываем матрицу коэффициентов A.
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                std::cin >> A[i][j];
                if (!std::cin) {
                    std::cerr << "Ошибка: не удалось считать элемент
матрицы A.\n";
                    return 1;
                }
            }
        }

        // Считываем вектор правых частей b.
        for (int i = 0; i < n; ++i) {
            std::cin >> b[i];
            if (!std::cin) {
                std::cerr << "Ошибка: не удалось считать элемент вектора
b.\n";
                return 1;
            }
        }
    }
}

```

```

    }

    // Создаём решатель.
    GaussianSolver solver(1e-12);

    // Решаем систему.
    const GaussResult result = solver.solve(A, b);

    // Печатаем результат.
    printGaussResult(result);
}
catch (const std::exception& ex) {
    std::cerr << "Исключение: " << ex.what() << "\n";
    return 1;
}

return 0;
}

```

Название файла: test.cpp

```

#include "gauss.hpp"

#include <cassert>
#include <chrono>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

// Простой мини-фреймворк для тестирования.
// Мы не используем сторонние библиотеки,
// чтобы файл можно было легко собрать "как есть".

// Счётчики тестов.
static int g_totalTests = 0;
static int g_passedTests = 0;

// Проверка условия.
// Если условие ложно - бросаем исключение.

void require(bool condition, const std::string& message)
{
    if (!condition) {
        throw std::runtime_error(message);
    }
}

// Проверка чисел на приблизительное равенство.

bool approxEqual(double a, double b, double eps = 1e-9)
{
    return std::fabs(a - b) <= eps;
}

```

```

// Проверка двух векторов на приблизительное равенство.

bool vectorApproxEqual(const std::vector<double>& a,
                      const std::vector<double>& b,
                      double eps = 1e-9)
{
    if (a.size() != b.size()) {
        return false;
    }

    for (std::size_t i = 0; i < a.size(); ++i) {
        if (!approxEqual(a[i], b[i], eps)) {
            return false;
        }
    }

    return true;
}

// Умножение матрицы A на вектор x.
// Нужно для проверки невязки в тестах.

std::vector<double> multiply(const std::vector<std::vector<double>>& A,
                           const std::vector<double>& x)
{
    const int n = static_cast<int>(A.size());
    std::vector<double> result(n, 0.0);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < static_cast<int>(A[i].size()); ++j) {
            result[i] += A[i][j] * x[j];
        }
    }

    return result;
}

// Максимальная норма невязки:
//  $\max_i |(A \cdot x)_i - b_i|$ 

double residualInfNorm(const std::vector<std::vector<double>>& A,
                      const std::vector<double>& x,
                      const std::vector<double>& b)
{
    const std::vector<double> Ax = multiply(A, x);

    double maxValue = 0.0;
    for (std::size_t i = 0; i < b.size(); ++i) {
        maxValue = std::max(maxValue, std::fabs(Ax[i] - b[i]));
    }
    return maxValue;
}

```

```

// Генерация единичной матрицы размера n.

std::vector<std::vector<double>> makeIdentityMatrix(int n)
{
    std::vector<std::vector<double>> A(n, std::vector<double>(n, 0.0));
    for (int i = 0; i < n; ++i) {
        A[i][i] = 1.0;
    }
    return A;
}

// Генерация диагональной матрицы.
// Диагональ: 1, 2, 3, ..., n
//
// Такой тест полезен для больших размеров,
// потому что система хорошо обусловлена,
// а ожидаемое решение легко проверить.

std::vector<std::vector<double>> makeDiagonalMatrix(int n)
{
    std::vector<std::vector<double>> A(n, std::vector<double>(n, 0.0));
    for (int i = 0; i < n; ++i) {
        A[i][i] = static_cast<double>(i + 1);
    }
    return A;
}

// Генерация матрицы Гильберта:
//  $H(i, j) = 1 / (i + j + 1)$ , если индексация с 0.
//
// Матрицы Гильберта - классический пример плохо обусловленных
// матриц. Они нужны для проверки численной устойчивости метода.

std::vector<std::vector<double>> makeHilbertMatrix(int n)
{
    std::vector<std::vector<double>> H(n, std::vector<double>(n, 0.0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            H[i][j] = 1.0 / static_cast<double>(i + j + 1);
        }
    }
    return H;
}

// Удобный запуск одного теста.

template <typename Func>
void runTest(const std::string& testName, Func testFunc)
{
    ++g_totalTests;

    try {
        testFunc();
        ++g_passedTests;
        std::cout << "[OK]    " << testName << "\n";
    }
}

```

```

    }
    catch (const std::exception& ex) {
        std::cout << "[FAIL] " << testName << " : " << ex.what() << "\n";
    }
    catch (...) {
        std::cout << "[FAIL] " << testName << " : неизвестная ошибка\n";
    }
}

// Базовый тест: обычная система 2x2 с единственным решением.

void testBasicUnique()
{
    std::vector<std::vector<double>> A = {
        {2.0, 1.0},
        {5.0, 7.0}
    };
    std::vector<double> b = {11.0, 13.0};

    GaussianSolver solver;
    const GaussResult result = solver.solve(A, b);

    require(result.type == SolutionType::Unique, "Ожидалось единственное
решение.");
    require(!result.isDegenerate, "Матрица не должна быть вырожденной.");

    const std::vector<double> expected = {64.0 / 9.0, -29.0 / 9.0};
    require(vectorApproxEqual(result.x, expected, 1e-9), "Неверное
решение 2x2.");
}

// Тест, где без перестановки строк метод мог бы сломаться:
// в первом элементе стоит 0, поэтому нужен выбор ведущего
// элемента и обмен строк.

void testPivotingIsReallyUsed()
{
    std::vector<std::vector<double>> A = {
        {0.0, 2.0},
        {1.0, 3.0}
    };
    std::vector<double> b = {4.0, 5.0};

    GaussianSolver solver;
    const GaussResult result = solver.solve(A, b);

    require(result.type == SolutionType::Unique, "Ожидалось единственное
решение.");
    require(!result.isDegenerate, "Матрица не должна быть вырожденной.");

    const std::vector<double> expected = {-1.0, 2.0};
    require(vectorApproxEqual(result.x, expected, 1e-9),
        "Неверное решение при необходимости выбора ведущего
элемента.");
}

```

```

// Тест на несовместную систему.
// Пример:
//  $x + y = 2$ 
//  $2x + 2y = 5$ 
//
// Вторая строка противоречит первой, решений нет.

void testNoSolution()
{
    std::vector<std::vector<double>> A = {
        {1.0, 1.0},
        {2.0, 2.0}
    };
    std::vector<double> b = {2.0, 5.0};

    GaussianSolver solver;
    const GaussResult result = solver.solve(A, b);

    require(result.type == SolutionType::None, "Ожидалось отсутствие
решений.");
    require(result.isDegenerate, "Матрица должна быть вырожденной.");
    require(result.rankA < result.rankAugmented, "Должно быть rank(A) <
rank([A|b]).");
}

// Тест на бесконечно много решений.
// Пример:
//  $x + y = 2$ 
//  $2x + 2y = 4$ 
//
// Вторая строка линейно зависит от первой.

void testInfiniteSolutions()
{
    std::vector<std::vector<double>> A = {
        {1.0, 1.0},
        {2.0, 2.0}
    };
    std::vector<double> b = {2.0, 4.0};

    GaussianSolver solver;
    const GaussResult result = solver.solve(A, b);

    require(result.type == SolutionType::Infinite, "Ожидалось бесконечно
много решений.");
    require(result.isDegenerate, "Матрица должна быть вырожденной.");
    require(result.rankA == result.rankAugmented, "Для совместной
вырожденной системы ранги должны совпадать.");

    // Проверяем, что найденное частное решение действительно
    удовлетворяет системе.
    const double residual = residualInfNorm(A, result.x, b);
    require(residual < 1e-9, "Частное решение не удовлетворяет
системе.");
}

```

```

// Тест на единичной матрице.
// Ожидаем, что решение равно правой части.

void testIdentityMatrix()
{
    const int n = 5;
    std::vector<std::vector<double>> A = makeIdentityMatrix(n);
    std::vector<double> b = {10.0, -2.0, 3.5, 7.0, 0.25};

    GaussianSolver solver;
    const GaussResult result = solver.solve(A, b);

    require(result.type == SolutionType::Unique, "Ожидалось единственное
решение.");
    require(!result.isDegenerate, "Единичная матрица невырождена.");
    require(vectorApproxEqual(result.x, b, 1e-12), "Для единичной матрицы
x должно совпадать с b.");
}

// Тест на матрице Гильберта.
//
// Важно:
// матрица Гильберта очень плохо обусловлена.
// Поэтому проверять точное совпадение координат решения
// не всегда разумно. Гораздо правильнее проверять невязку.
//
// Здесь мы задаём x_true = [1, 1, ..., 1],
// считаем b = H * x_true,
// затем решаем систему и проверяем:
// 1) что решение найдено,
// 2) что невязка маленькая.

void testHilbertMatrix(int n, double maxResidualAllowed)
{
    std::vector<std::vector<double>> H = makeHilbertMatrix(n);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(H, xTrue);

    GaussianSolver solver(1e-14);
    const GaussResult result = solver.solve(H, b);

    require(result.type == SolutionType::Unique,
        "Для матрицы Гильберта ожидалось единственное решение.");

    const double residual = residualInfNorm(H, result.x, b);
    require(residual < maxResidualAllowed,
        "Слишком большая невязка на матрице Гильберта.");
}

// Большой тест на диагональной матрице.
// Такой случай удобен тем, что:
// - матрица точно невырождена,
// - решение известно заранее,
// - тест не такой "ядовитый", как случайная плотная матрица.

```

```

void testLargeDiagonalMatrix(int n)
{
    std::vector<std::vector<double>> A = makeDiagonalMatrix(n);

    // Возьмём истинное решение x = [1, 1, ..., 1].
    std::vector<double> xTrue(n, 1.0);

    // Тогда b просто равен диагонали.
    std::vector<double> b(n, 0.0);
    for (int i = 0; i < n; ++i) {
        b[i] = static_cast<double>(i + 1);
    }

    GaussianSolver solver;

    const auto start = std::chrono::steady_clock::now();
    const GaussResult result = solver.solve(A, b);
    const auto finish = std::chrono::steady_clock::now();

    const std::chrono::duration<double> elapsed = finish - start;

    require(result.type == SolutionType::Unique, "Ожидалось единственное
решение для большой диагональной матрицы.");
    require(!result.isDegenerate, "Диагональная матрица с ненулевой
диагональю невырождена.");

    const double residual = residualInfNorm(A, result.x, b);
    require(residual < 1e-9, "Слишком большая невязка на большой
диагональной матрице.");

    std::cout << "        Размер n = " << n
              << ", время = " << elapsed.count() << " сек.\n";
}

// Главная функция тестов.

int main()
{
    std::cout << "Запуск тестов метода Гаусса\n";
    std::cout << "-----\n";

    runTest("Базовый случай 2x2", testBasicUnique);
    runTest("Проверка выбора ведущего элемента",
testPivotingIsReallyUsed);
    runTest("Несовместная система", testNoSolution);
    runTest("Бесконечно много решений", testInfiniteSolutions);
    runTest("Единичная матрица", testIdentityMatrix);

    // Матрицы Гильберта.
    runTest("Матрица Гильберта 5x5", []() {
        testHilbertMatrix(5, 1e-10);
    });

    runTest("Матрица Гильберта 8x8", []() {
        testHilbertMatrix(8, 1e-8);
    });
}

```



```

// Большой тест 1000x1000.
// Он уже вполне серьёзный по памяти и времени,
// но обычно ещё выполним на обычной машине.
runTest("Большая диагональная матрица 1000x1000", []() {
    testLargeDiagonalMatrix(1000);
});

// Очень большой тест 10000x10000.
//
// ВАЖНО:
// Для плотного метода Гаусса такой размер уже крайне тяжёлый:
// - память очень большая,
// - время у классического метода  $O(n^3)$ .
//
// Поэтому этот тест по умолчанию отключён.
// Его можно включить при компиляции макросом:
// -DRUN_VERY_LARGE_TESTS
//
// Это честнее, чем делать вид, что такой тест "лёгкий".
// На практике для 10000x10000 плотный Гаусс - это уже
// очень тяжёлая задача.
#ifdef RUN_VERY_LARGE_TESTS
    runTest("Очень большая диагональная матрица 10000x10000", []() {
        testLargeDiagonalMatrix(10000);
    });
#else
    std::cout << "[SKIP] Очень большая диагональная матрица 10000x10000 "
                << "(отключено по умолчанию, включается через - "
DRUN_VERY_LARGE_TESTS) "\n";
#endif

    std::cout << "-----\n";
    std::cout << "Пройдено тестов: " << g_passedTests
                << " из " << g_totalTests << "\n";

    if (g_passedTests == g_totalTests) {
        std::cout << "Все тесты успешно пройдены.\n";
        return 0;
    }

    std::cout << "Есть непройденные тесты.\n";
    return 1;
}

```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Таблица тестирования задачи:

№	Ввод	Вывод	Время	Память
1	n=2, матрица: 2 1 / 5 7, вектор: 11 13	Единственное решение, матрица невырождена, $x =$ [7,111111; - 3,222222]	6,221 мс	7656 КБ
2	n=2, матрица: 0 2 / 1 3, вектор: 4 5	Единственное решение, матрица невырождена, $x =$ [-1; 2]	6,246 мс	7704 КБ
3	n=2, матрица: 1 1 / 2 2, вектор: 2 5	Решений нет, матрица вырождена	5,660 мс	6656 КБ
4	n=2, матрица: 1 1 / 2 2, вектор: 2 4	Бесконечно много решений, матрица вырождена, одно частное решение: $x = [2; 0]$	6,005 мс	7224 КБ
5	n=5, единичная матрица, вектор: 10 -2 3,5 7 0,25	Единственное решение, матрица невырождена, $x =$ [10; -2; 3,5; 7; 0,25]	6,393 мс	5704 КБ
6	n=5, матрица Гильберта, вектор правых	Единственное решение, матрица невырождена, $x \approx$	6,673 мс	5720 КБ

	частей сформирован по известному решению	[1; 1; 1; 1; 1], невязка мала		
7	n=8, матрица Гильберта, вектор правых частей сформирован по известному решению	Единственное решение, матрица невырождена, $x \approx$ [1; 1; 1; 1; 1; 1; 1; 1], невязка мала	6,139 мс	7328 КБ
8	n=1000, диагональная матрица, вектор: 1 2 3 ... 1000	Единственное решение, матрица невырождена, $x =$ [1; 1; 1; ...; 1]	35,509 мс	23296 КБ
9	n=10000, диагональная матрица, вектор: 1 2 3 ... 10000	Единственное решение, матрица невырождена, $x =$ [1; 1; 1; ...; 1]	2902,131 мс	1570572 КБ

Таблица 1 – Тестирование алгоритма.